

Core ML

Hardik Jain

AIM

To design a long context sequence model and conduct tests on different types of Attention, Positional Embeddings and convolutional layers

Contents

1	Introduction	2
2	Setup	2
2.1	Dataset	2
2.2	Hardware	2
2.3	Parameters	2
2.4	Training	3
2.5	Evaluation Metrics	3
2.6		3
3	Baseline Transformer	3
3.1	Architecture	3
3.2	Baseline Configuration	4
3.3	Baseline Results	4
3.4	Reference Point	4
4	Attention Variants	4
4.1	Results	5
4.2	Why these results	5
5	Positional Embeddings and Extrapolation	5
5.1	Extrapolation Results	7
5.2	Results	7
6	Convolution + Attention Hybrid	7
6.1	Results	8
6.2	Why these results	9
7	Problems faced during implementation	9
8	Conclusion	10

1 Introduction

Long context sequence modeling remains a major challenge in modern deep learning systems. Standard Transformer architectures on full self attention scale quadratically with the context window and there are limitations to positional encoding being extrapolated.

Recent newer mechanism for both attention and positional encoding tries to make the model more efficient for longer context windows and encodings which can be extrapolated for higher context.

In this assignment, I implement a Transformer having interchangeable attention mechanisms, positional embeddings, and convolutional components like a plug and play system. I am using WikiText-2 dataset to evaluate model,efficiency, peak gpu use, and extrapolation in different types of positional encodings.

The order in which this is done is:

- Implementation of multiple efficient attention variants.
- Extrapolation of different types of positional encodings.
- Implementation of different types of Convolution Hybrids.
- View on theoretical versus practical outputs.

2 Setup

2.1 Dataset

I used the WikiText-2 dataset. The data set consists of long Wikipedia articles. The tokenzier I used was tiktoken gpt2.

2.2 Hardware

I used PyTorch for everything and I trained the model on MPS(Apple Metal Performance Shaders) instead of CUDA because using apple silicon was more comfortable for me. As all variants were trained on the same hardware it did not matter because we see only relative performance.

2.3 Parameters

The architecture used in this assignment is a decoder only Transformer language model with causal masking. The primary "hyperparameters" used for most experiments are shown below:

Parameter	Value
Sequence Length	1024
Embedding Dimension (d_{model})	256
Number of Heads	2
Number of Layers	4
Batch Size	2
Dropout	0.1
Learning Rate	3×10^{-4}
Optimizer	AdamW
Epochs	10

Table 1: Primary Training Hyperparameters

2.4 Training

The models were trained using AdamW optimizer with cosine learning rate scheduling. I also used gradient clipping to prevent exploding gradients and multiple times. If I did not use this Sliding Window attention was giving OOM errors.

2.5 Evaluation Metrics

For all experiments, the following metrics were used to see which things are better than others:

- Training Loss
- Validation Loss
- Validation Perplexity
- Peak GPU Memory Usage
- Training Time per Epoch
- Inference Latency
- Throughput (tokens/sec)
- Training Stability (variance of training loss)

2.6

The tests were divided into three categories:

1. Attention Variant Comparison
2. Positional Embedding's Extrapolation Comparison
3. Convolution + Attention Hybrid Comparison

For attention, I used different attention mechanisms while using position embedding RoPE for all of them.

For positional embedding, I used the best performing attention mechanism (MQA) and doing Extrapolation on the data.

For Convolution hybrid I used MQA+ALiBi to evaluate.

3 Baseline Transformer

3.1 Architecture

The baseline model I used based on the standard Transformer proposed in "Attention Is All You Need". The model uses causal self attention for next token prediction. Each Transformer block consists of:

- Multi Head Self Attention
- Dropout
- Layer Normalization
- Feedforward Network

3.2 Baseline Configuration

The baseline model was trained using the following configuration:

Parameter	Value
Attention Type	Standard Self Attention
Positional Embedding	RoPE
Convolution Type	None

Table 2: Baseline Transformer Configuration

3.3 Baseline Results

Metric	Value
Train Loss	3.8035
Validation Loss	5.0122
Validation Perplexity	150.2314
Training Stability	0.053947
Training Time / Epoch	106.16 sec
Inference Latency	4.82ms
Throughput	212586.89 tokens/sec
Peak GPU Memory	451.34 MB

Table 3: Baseline Transformer Performance

3.4 Reference Point

This is a reference point for all later attention variants, positional embeddings, and convolutional hybrids.

”Also an important thing to note is that the Throughput is unusually large and that may be due to calculating it in batched tokens and also due to the fact that we using MPS. But we just need to make comparisons so this is fine.”

4 Attention Variants

To study the effect of efficient attention mechanisms on long-context language modeling, multiple attention variants were implemented and compared against standard self-attention. All experiments in this section were conducted using identical training settings while varying only the attention mechanism.

The evaluated attention variants include:

- Standard Self Attention
- Sliding Window Attention
- Linear Attention
- Multi Query Attention (MQA)

4.1 Results

Attention Type	Train Loss	Val Loss	Perplexity
Standard	3.80	5.01	150.23
Sliding Window	4.01	5.26	192.91
Linear Attention	4.16	5.36	212.37
MQA	3.83	5.02	150.82

Table 4: Loss Comparison

Attention Type	Latency	Throughput	Memory	Time/Epoch
Standard	4.82 ms	212586 tok/s	451.34 MB	106.16 s
Sliding Window	13.50 ms	75839 tok/s	441.39 MB	167.58 s
Linear Attention	19.27 ms	53137 tok/s	441.45 MB	292.74 s
MQA	4.72 ms	216781 tok/s	449.85 MB	107.78 s

Table 5: Efficiency Comparison

4.2 Why these results

If we see the memory, there was a very slight difference so we cant differentiate model efficiency based on this.

Also the standard model beat both sliding window and linear attention in efficiency although the reverse was expected. I think this might have happened due to very optimised pytorch standard matrix multiplications. Also in Sliding Window padding is required which is not very optimized on MPS and in linear attention also cumsum is used not optimized as good as standard multiplications. And we are using just 1024 context window so $O(N^2)$ vs $O(N*w)$ are not that far away to create a difference because the inner constants are bigger in sliding window and linear attentions.

If we see MQA we see that it was slightly better in latency from standard type and it performed normally unlike Sliding Window and Linear Attention. This was because here normal operations are being done as in standard we are just reusing the same Key and value matrices. So it improves more with increase in context window.

So overall best attention type was MQA as it would scale better with context size.

A slight overfitting can also be seen as there is some difference in the train loss and val loss. This might be because the dataset is not that large.

5 Positional Embeddings and Extrapolation

After identifying MQA as the best-performing attention variant from the previous experiments, positional embedding methods were evaluated using the same attention configuration.

The primary goal of these experiments was to study how different positional encoding techniques extrapolate beyond their training context length. All models in this section were trained using a sequence length of 512 tokens and later evaluated on longer unseen context lengths of 1024 and 2048 tokens.

The following positional embedding methods were compared:

- Sinusoidal Positional Embeddings
- Rotary Positional Embeddings (RoPE)
- ALiBi Positional Biases

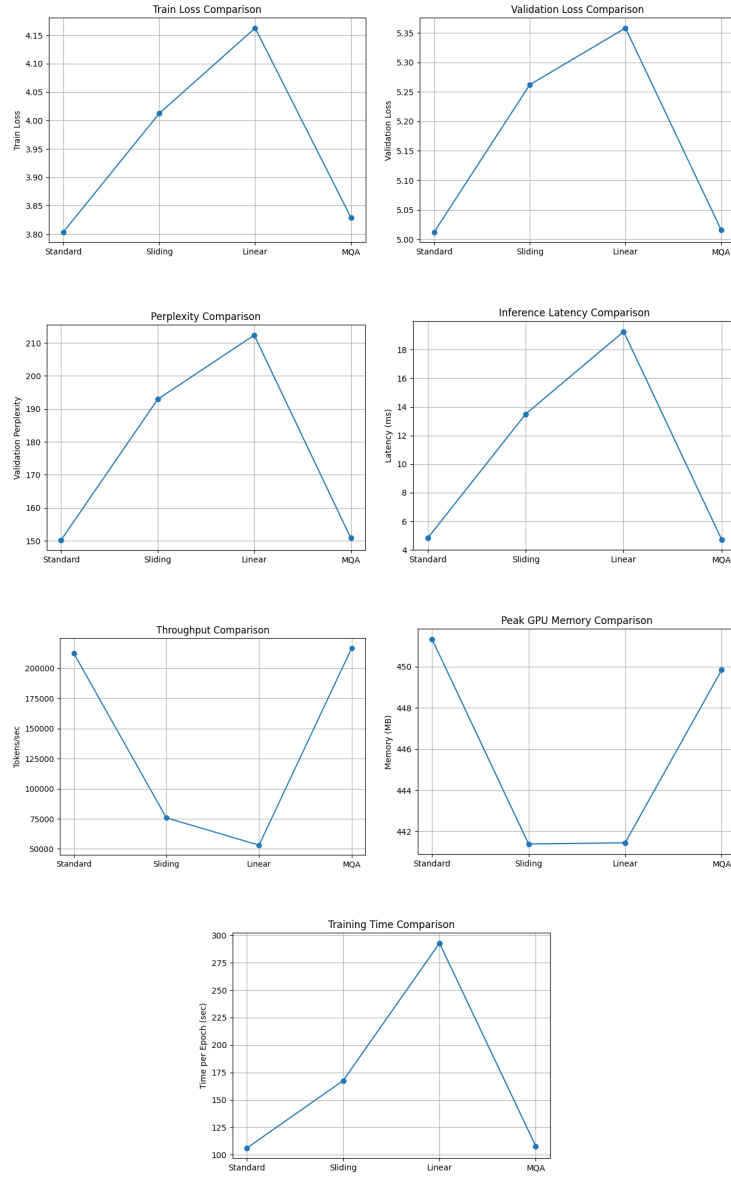


Figure 1: Comparison of attention variants across training loss, validation loss, perplexity, inference latency, throughput, GPU memory usage, and training time per epoch.

5.1 Extrapolation Results

Embedding	PPL-512	PPL-1024	PPL-2048	Loss-512	Loss-1024	Loss-2048
Sinusoidal	191.17	256.87	322.01	5.25	5.55	5.77
RoPE	154.57	171.33	207.29	5.04	5.14	5.33
ALiBi	154.20	152.91	152.08	5.04	5.03	5.02

Table 6: Extrapolation Performance of Positional Embeddings

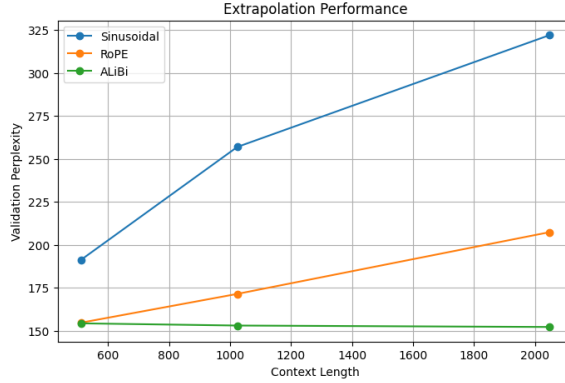


Figure 2: Extrapolation behavior of Sinusoidal, RoPE, and ALiBi positional embeddings from 512 to 2048 context window

5.2 Results

The extrapolation experiments have clear differences between different positional embedding.

The sinusoidal embedding failed by a lot in extrapolation. This was because the embeddings are fixed for positions till 512 and the model has not seen the next positions so the embedding could not be calculated accurately by the model on extrapolation.

RoPE generalizes much better than sinusoidal because it relies on the relative angle between the Query and Key matrices of the embedding of each word. For example it understand how far are 2 words not that one is at 10 and other 20. It fails because it is only trained such that words can be maximum 512 distance apart. So while running on 2048 context size, it leads to higher loss.

ALiBi was the best one here because of it's method of not storing any angles or adding any embeddings based on relative distance. It just adds a penalty based on the the distnace which scales linearly. So even though it has not seen on 512 context how far can the words be, it can just adjust the penalty based on the context size. So it remains super flat the entire extrapolation.

Also we can not perform extrapolation on Relative Positional Encoding becuase it was a fixed learned marix which can't be changes according to different context size.

6 Convolution + Attention Hybrid

We are using MQA + ALiBi as a base here because it performed the best in the previous tests.

The following are compared:

- MQA + ALiBi (No Convolution)

- Convolution Before Attention
- Gated Convolutional Feedforward Network

Conv1D layer before each attention block is applied for the 2nd part.

The gated convolutional feedforward uses a gated convolutional module using causal convolution.

6.1 Results

Architecture	Train Loss	Val Loss	Perplexity
No Conv	3.55	5.04	154.20
Conv Before Attention	3.57	5.07	159.82
Gated Conv FFN	3.47	5.23	187.63

Table 7: Language Modeling Performance of Convolution Hybrid Architectures

Architecture	Latency	Throughput	Memory	Time/Epoch
No Conv	2.73 ms	187226 tok/s	468 MB	111.62 s
Conv Before Attention	9.70 ms	105610 tok/s	457 MB	112.13 s
Gated Conv FFN	8.86 ms	115549 tok/s	453 MB	108.63 s

Table 8: Efficiency Comparison of Convolution Hybrid Architectures

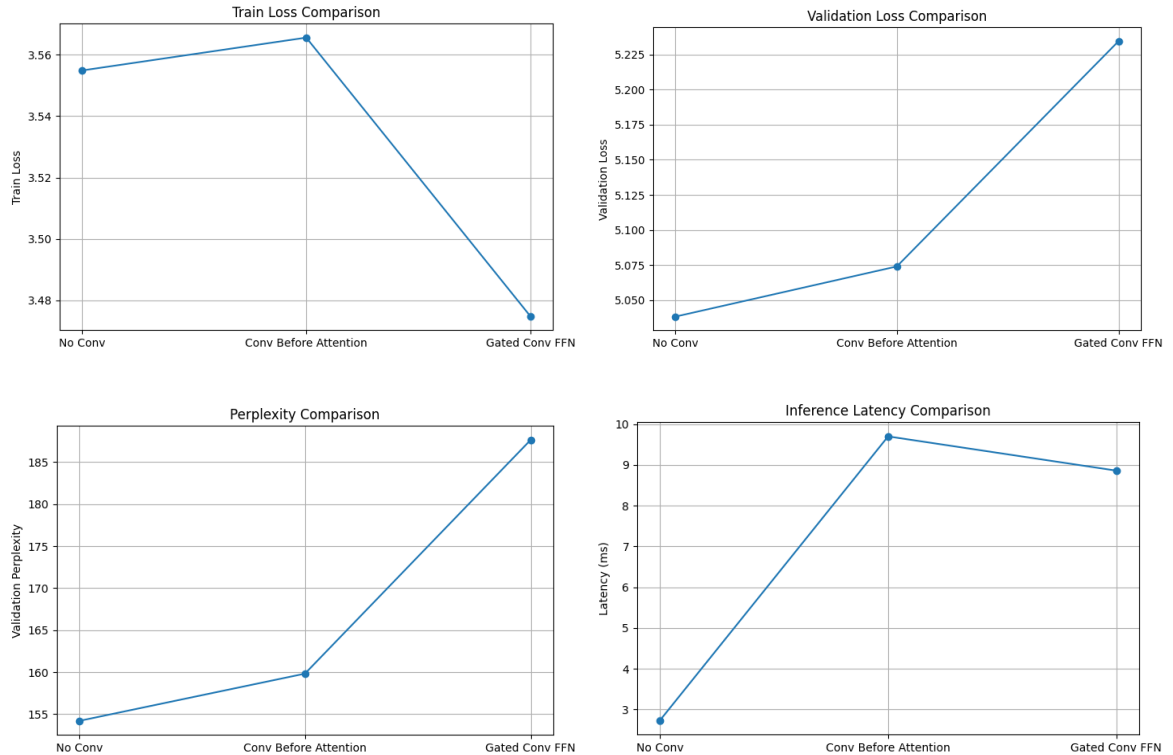


Figure 3: Loss, perplexity, and latency comparison of convolution hybrid.

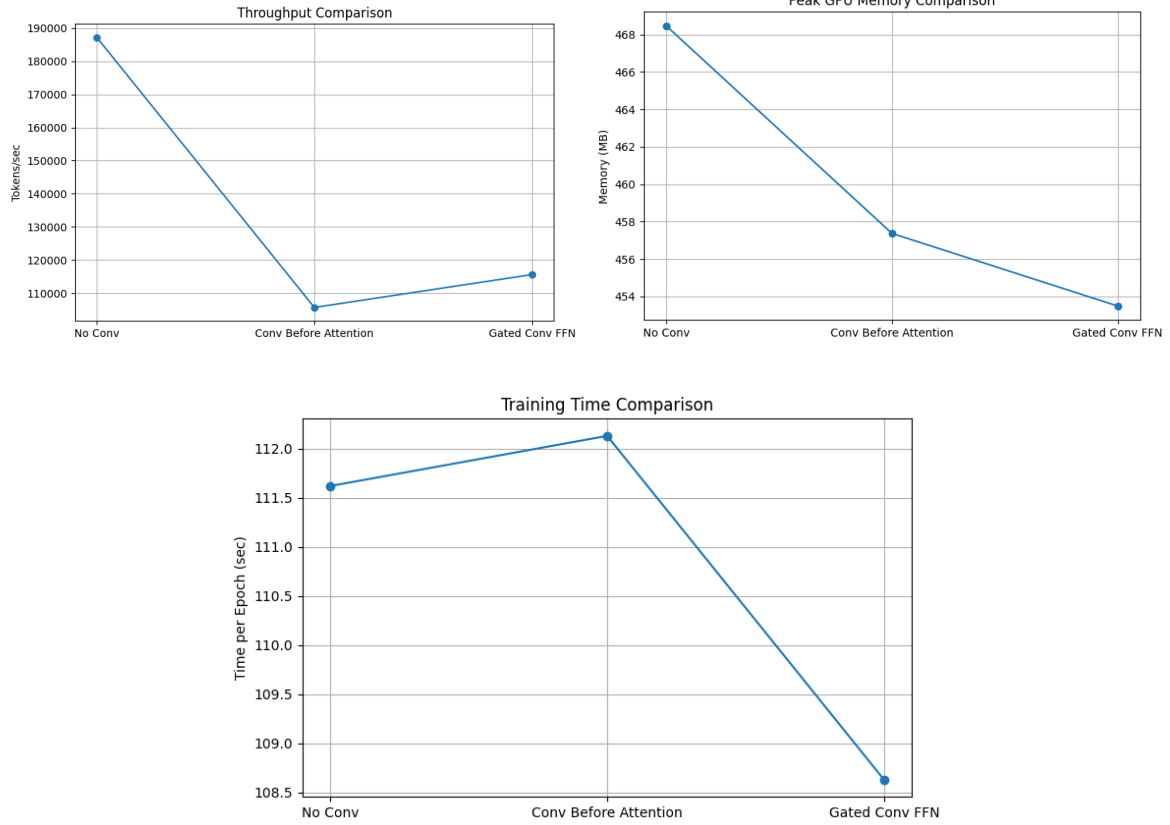


Figure 4: Throughput, memory usage, and training time comparison of convolution hybrid.

6.2 Why these results

The convolution hybrid showed that adding convolutional did not improve validation perplexity compared to the baseline MQA + ALiBi.

The convolution before attention model achieved slightly lower training loss while maintaining validation performance relatively close to baseline. But the inference latency increased and throughput decreased due to the additional operations.

The gated convolutional feedforward architecture achieved the lowest training loss but also produced the worst validation perplexity. This means the model was overfitting a lot and did not perform better than the baseline model.

The reason why convolutional layers did not perform well maybe because that convolutional make each word learn about its nearby words more than the words far away from them, so the whole context is difficult to take into each embedding of each word.

7 Problems faced during implementation

- During implementation of RoPE, a mismatch was there between the sinusoidal frequency layout and the vector rotation method. The code used a split half frequency arrangement while applying pairwise rotations using stride 2 indexing, which produced incorrect positional rotations. So matching both ways the values were processed was important.
- A major issue occurred when the convolution layers used symmetric padding, which unintentionally allowed future tokens to leak into the current token representation, basically no masking of the words ahead of each word.

This caused very very low training loss values and invalid evaluation. So it was corrected by masking the next words of each word.

- During extrapolation, positional embedding were dependent directly on the training context length, causing errors during extrapolation because the context size was changing. I corrected this by allocating a MAX length of the context window and the code only used till the required length in extrapolation.

8 Conclusion

In this assignment, the tests show that choices for long context models involve tradeoffs between efficiency, extrapolation capability, memory usage. Among all the configurations, the combination of MQA attention with ALiBi positional encoding provided the best overall performance and scalability in this setup. We created a plug and play system in code so we can test any attention type with any positional encoding with any convolutional hybrid.